

1.Especificação de requisitos

Documento de Especificação de Requisitos – BoardGame Bar

A presente solicitação tem como objetivo a modelagem e o desenvolvimento de um sistema de banco de dados relacional para gerenciar as operações do BoardGame Bar, um estabelecimento que integra os serviços de hamburgueria artesanal, cafeteria e locadora de jogos de tabuleiro. O modelo de negócio exige um controle rigoroso que una o salão de atendimento e os bastidores operacionais em uma única plataforma, garantindo a integridade das informações desde a chegada do cliente até o fechamento financeiro do mês. O fluxo operacional se inicia no salão, onde precisamos cadastrar nossos clientes e gerenciar o fluxo de ocupação do espaço. O sistema deve ser capaz de registrar reservas futuras, alocando clientes específicos para mesas com capacidades determinadas, além de controlar a jornada dos nossos funcionários, identificando qual garçom ou atendente é responsável por cada atendimento no dia a dia.

Quando um cliente ocupa uma mesa, inicia-se o processo de venda através de uma comanda. Esta comanda é o coração do nosso faturamento e precisa possuir uma estrutura flexível e robusta, capaz de unificar duas frentes de consumo distintas. A primeira frente é a gastronômica, onde o cliente pode solicitar produtos do cardápio, como lanches e bebidas, havendo também a necessidade imperativa de registrar a inclusão de insumos adicionais, como uma porção extra de bacon ou queijo, cobrados à parte. A segunda frente é a de entretenimento, que exige que a comanda registre o aluguel de jogos de tabuleiro do nosso acervo, cuja cobrança não é fixa, mas sim calculada com base no valor da hora de aluguel de cada título específico. O sistema deve consolidar todas essas informações, aplicando eventuais descontos percentuais concedidos pela gerência e calculando a taxa de gorjeta para compor o valor total de consumo da mesa.

Para que a frente gastronômica funcione sem falhas, o banco de dados deve gerenciar todo o nosso back-office de forma inteligente. Precisamos de um controle de compras de estoque que registre as notas fiscais emitidas por nossos fornecedores e alimente nosso inventário de insumos brutos, como quilos de batata ou litros de xarope. É fundamental que o sistema compreenda a composição dos nossos pratos através de fichas técnicas, estabelecendo a relação exata de quais insumos, e em quais frações ou quantidades, são necessários para montar cada produto vendido. Dessa forma, garantimos que os custos operacionais estejam sempre mapeados e o estoque reflita a realidade da nossa cozinha de forma automatizada a cada saída registrada nas comandas.

Por fim, a administração do BoardGame Bar necessita de extrações de dados complexas para a tomada de decisões estratégicas, exigindo relatórios gerenciais que cruzem múltiplas áreas da empresa simultaneamente. O primeiro relatório essencial deve fornecer um extrato completo e detalhado das comandas ativas e fechadas, cruzando simultaneamente os dados do cliente titular, os detalhes da mesa ocupada, o funcionário responsável pelo atendimento e a listagem completa de todos os produtos consumidos e jogos alugados. O segundo levantamento deve mapear a nossa agenda de ocupação, conectando os perfis dos clientes às suas respectivas reservas e às mesas designadas, permitindo uma visão clara da nossa capacidade de atendimento diária. O terceiro e mais crítico requisito analítico

é a geração de um demonstrativo financeiro mensal, um relatório consolidado que apure o lucro líquido da empresa ao cruzar o faturamento limpo extraído das comandas, já deduzidos os descontos, subtraindo os custos totais das compras de insumos junto aos fornecedores e as despesas com a folha de pagamento dos funcionários ativos no período.

2 Modelagem MER

A. Entidades Independentes (Fortes)

São as entidades que não dependem de nenhuma outra para existir no sistema.

- **Cliente**
 - `id_cliente` (Inteiro, Chave Primária - PK)
 - `nome_completo` (Texto)
 - `cpf` (Texto, Único)
 - `telefone` (Texto)
 - `email` (Texto)
- **Funcionario**
 - `id_funcionario` (Inteiro, Chave Primária - PK)
 - `nome_funcionario` (Texto)
 - `cpf_funcionario` (Texto, Único)
 - `cargo` (Texto)
 - `data_admissao` (Data)
 - `status_funcionario` (Texto)
 - `salário` (Numérico)
- **Mesa**
 - `id_mesa` (Inteiro, Chave Primária - PK)
 - `numero_mesa` (Inteiro, Único)
 - `capacidade_lugares` (Inteiro)
 - `status_mesa` (Texto)
- **Fornecedor**
 - `id_fornecedor` (Inteiro, Chave Primária - PK)
 - `nome_fornecedor` (Texto)
 - `cnpj` (Texto, Único)
 - `telefone_fornecedor` (Texto)
- **Produto**
 - `id_produto` (Inteiro, Chave Primária - PK)
 - `nome_produto` (Texto)
 - `categoria_produto` (Texto)
 - `preco_venda_tabela` (Numérico)
- **Insumo**
 - `id_insumo` (Inteiro, Chave Primária - PK)

- nome_insumo (Texto)
- qtd_estoque_atual (Numérico)
- unidade_medida (Texto)
- preco_unitario_atual (Numérico)
- **Jogo**
 - id_jogo (Inteiro, Chave Primária - PK)
 - nome_jogo (Texto)
 - categoria_jogo (Texto)
 - status_jogo (Texto)
 - aluguel_hora_atual (Numérico)

B. Entidades Dependentes (Com Chaves Estrangeiras)

São os registros de movimentação do dia a dia, que ligam as entidades fortes.

- **Reserva**
 - id_reserva (Inteiro, Chave Primária - PK)
 - id_cliente (Inteiro, Chave Estrangeira - FK)
 - id_mesa (Inteiro, Chave Estrangeira - FK)
 - data_hora_reserva (Data/Hora)
 - status_reserva (Texto)
 - qtd_pessoas_esperadas (Inteiro)
- **Compra_Estoque**
 - id_compra (Inteiro, Chave Primária - PK)
 - id_fornecedor (Inteiro, Chave Estrangeira - FK)
 - data_compra (Data)
 - valor_total_nota (Numérico)
- **Comanda**
 - id_comanda (Inteiro, Chave Primária - PK)
 - id_cliente (Inteiro, Chave Estrangeira - FK)
 - id_mesa (Inteiro, Chave Estrangeira - FK)
 - id_funcionario (Inteiro, Chave Estrangeira - FK)
 - data_abertura (Data/Hora)
 - valor_total_consumo (Numérico)
 - gorjeta (Numérico)
 - status_comanda (Texto)
 - desconto_percentual (Numérico)

C. Entidades Associativas (Tabelas Ponte / Relacionamentos N:M)

São tabelas criadas para resolver os relacionamentos de "Muitos para Muitos".

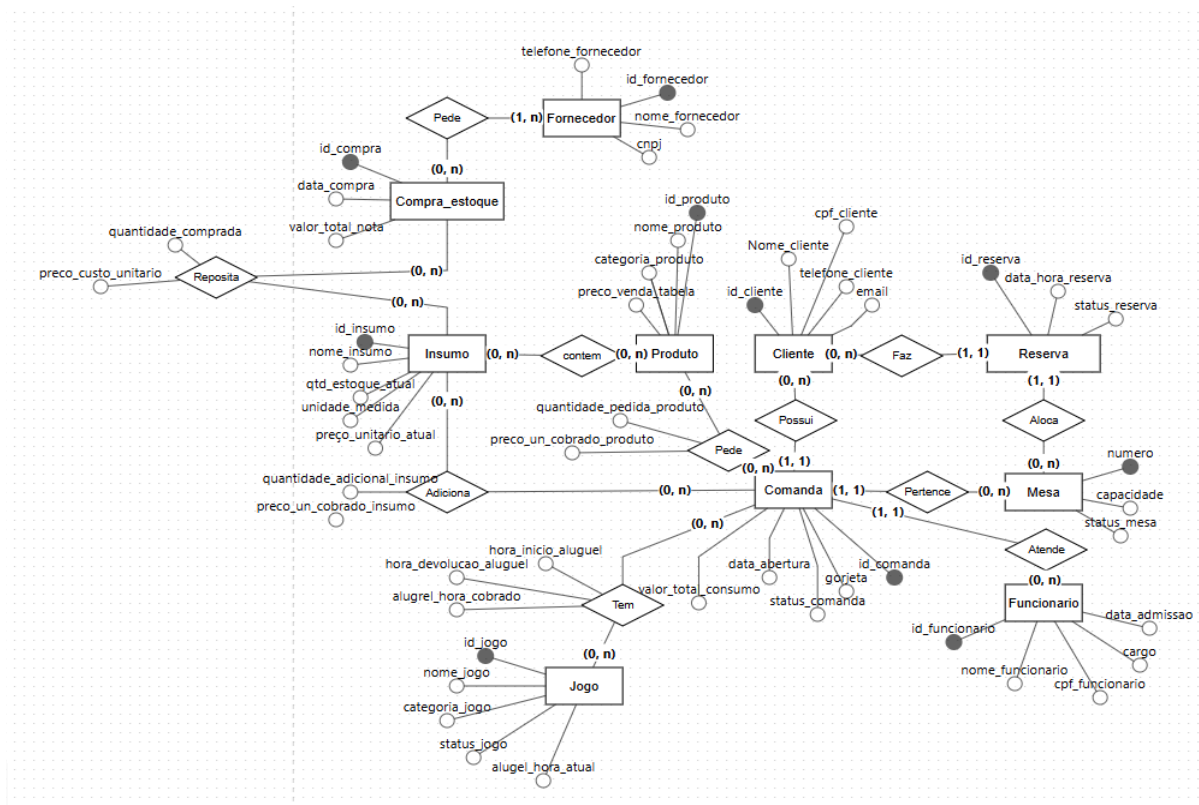
- **Item_Comanda_Produto** (Consumo de itens de cardápio)
 - id_comanda (Inteiro, PK Composta / FK)

- `id_produto` (Inteiro, PK Composta / FK)
- `quantidade_pedida_produto` (Inteiro)
- `preco_un_cobrado_produto` (Numérico)
- **Item_Comanda_Insumo** (Adicionais extras em um lanche, ex: "Extra Bacon")
 - `id_item_insumo` (Inteiro, Chave Primária - PK)*
 - `id_comanda` (Inteiro, Chave Estrangeira - FK)
 - `id_insumo` (Inteiro, Chave Estrangeira - FK)
 - `quantidade_adicional_insumo` (Inteiro)
 - `preco_un_cobrado_insumo` (Numérico)
 - `referencia_item_pai` (Inteiro) > *Nota técnica: Possui PK própria para permitir que o mesmo adicional seja lançado várias vezes em momentos diferentes na mesma comanda.*
- **Item_Comanda_Jogo** (Histórico de aluguel de jogos)
 - `id_comanda` (Inteiro, PK Composta / FK)
 - `id_jogo` (Inteiro, PK Composta / FK)
 - `hora_inicio_aluguel` (Data/Hora, PK Composta)
 - `hora_devolucao_aluguel` (Data/Hora)
 - `aluguel_hora_cobrado` (Numérico)
- **Ficha_Tecnica_Produto** (Receita de cada lanche para baixa de estoque)
 - `id_produto` (Inteiro, PK Composta / FK)
 - `id_insumo` (Inteiro, PK Composta / FK)
 - `quantidade_necessaria` (Numérico)

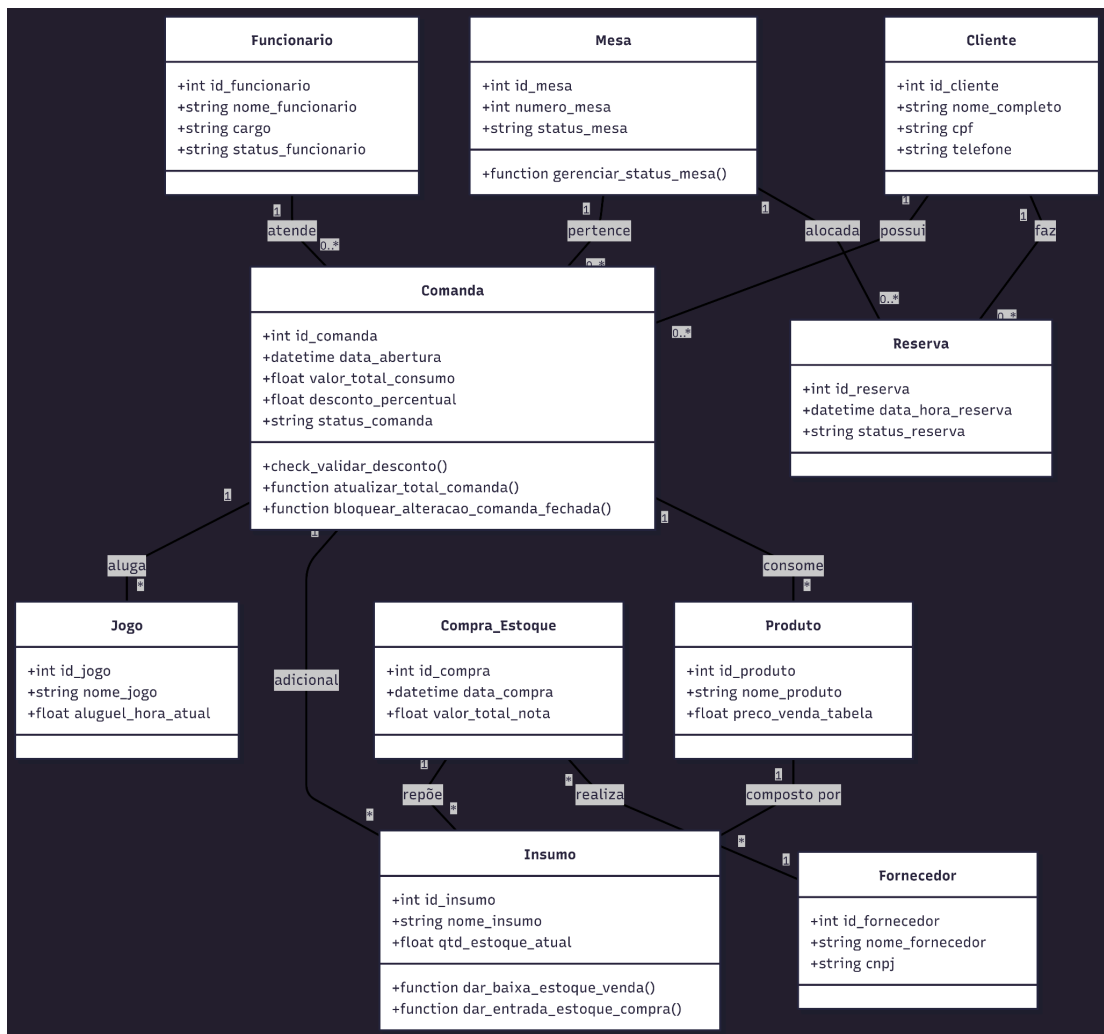
D. Relacionamentos e Cardinalidades (Regras de Negócio)

- **Atendimento (Comanda):** * Um **Cliente** pode abrir várias **Comandas** (1:N).
 - Uma **Mesa** pode receber várias **Comandas** (1:N).
 - Um **Funcionário** é responsável por abrir várias **Comandas** (1:N).
- **Consumo (Produtos e Adicionais):**
 - Uma **Comanda** possui vários **Produtos**, e um **Produto** é vendido em várias **Comandas** (N:M -> Resolvido por `Item Comanda Produto`).
 - Uma **Comanda** pode ter o consumo direto de vários **Insumos** como adicionais (N:M -> Resolvido por `Item_Comanda_Insumo`).
- **Locação (Jogos):**
 - Uma **Comanda** pode registrar o aluguel de vários **Jogos**, e um **Jogo** pode ser alugado em várias **Comandas** (N:M -> Resolvido por `Item_Comanda_Jogo`).
- **Agendamento (Reserva):**
 - Um **Cliente** faz várias **Reservas** e uma **Mesa** pode ser agendada em várias **Reservas** em datas diferentes (1:N).
- **Suprimentos (Estoque):**
 - Um **Fornecedor** atende a várias **Compras de Estoque** (1:N).
 - Um **Produto** é composto por vários **Insumos** (N:M -> Resolvido por `Ficha_Tecnica_Produto`).

DER:



3. Modelagem UML



4. Mapeamento DER para o Modelo Relacional

Regras de Mapeamento Aplicadas:

- Entidades Fortes viraram tabelas independentes.
- Relacionamentos 1:N (Um para Muitos): A chave primária do lado "1" migrou como Chave Estrangeira (FK) para a tabela do lado "N".
- Relacionamentos N:M (Muitos para Muitos): Viraram tabelas associativas próprias, importando as Chaves Primárias das tabelas originais para formar uma Chave Primária Composta.

Esquema Relacional:

- **Cliente** (**id_cliente** (PK), nome_completo, cpf, telefone, email)
- **Funcionario** (**id_funcionario** (PK), nome_funcionario, cpf_funcionario, cargo, data_admissao, status_funcionario)

- **Mesa** (**id_mesa (PK)**, numero_mesa, capacidade_lugares, status_mesa)
- **Fornecedor** (**id_fornecedor (PK)**, nome_fornecedor, cnpj, telefone_fornecedor)
- **Produto** (**id_produto (PK)**, nome_produto, categoria_produto, preco_venda_tabela)
- **Insumo** (**id_insumo (PK)**, nome_insumo, qtd_estoque_atual, unidade_medida, preco_unitario_atual)
- **Jogo** (**id_jogo (PK)**, nome_jogo, categoria_jogo, status_jogo, aluguel_hora_atual)

Tabelas Dependentes (Com Chaves Estrangeiras):

- **Reserva** (**id_reserva (PK)**, data_hora_reserva, status_reserva, qtd_pessoas_esperadas, id_cliente (FK), id_mesa (FK))
- **Compra_Estoque** (**id_compra (PK)**, data_compra, valor_total_nota, id_fornecedor (FK))
- **Comanda** (**id_comanda (PK)**, data_abertura, valor_total_consumo, gorjeta, status_comanda, desconto_percentual, id_cliente (FK), id_mesa (FK), id_funcionario (FK))

Tabelas Associativas (Resolução de Relacionamentos N:M):

- **Item_Comanda_Produto** (**id_comanda (PK)(FK)**, **id_produto (PK)(FK)**, quantidade_pedida_produto, preco_un_cobrado_produto)
- **Item_Comanda_Insumo** (**id_item_insumo (PK)**, id_comanda (FK), id_insumo (FK), quantidade_adicional_insumo, preco_un_cobrado_insumo, referencia_item_pai)
Nota: Recebeu PK própria ("Surrogate Key") para permitir a inserção do mesmo insumo adicional repetidas vezes na mesma comanda.
- **Item_Comanda_Jogo** (**id_comanda (PK)(FK)**, **id_jogo (PK)(FK)**, **hora_inicio_aluguel (PK)**, hora_devolucao_aluguel, aluguel_hora_cobrado)
- **Ficha_Tecnica_Produto** (**id_produto (PK)(FK)**, **id_insumo (PK)(FK)**, quantidade_necessaria)

5.Script de criação de tabelas e inserção (inserção inicial)

Criação

-- 1. TABELAS INDEPENDENTES (FORTES)

```
CREATE TABLE Cliente (
  id_cliente INT AUTO_INCREMENT PRIMARY KEY,
  nome_completo VARCHAR(100) NOT NULL,
  cpf VARCHAR(14) UNIQUE NOT NULL,
  telefone VARCHAR(15),
  email VARCHAR(100)
```

);

```
CREATE TABLE Funcionario (  
    id_funcionario SERIAL PRIMARY KEY,  
    nome_funcionario VARCHAR(100) NOT NULL,  
    cpf_funcionario VARCHAR(14) UNIQUE NOT NULL,  
    cargo VARCHAR(50) NOT NULL,  
    salario DECIMAL(10,2) NOT NULL DEFAULT 2500.00, -- Coluna necessária para o  
relatório financeiro  
    data_admissao DATE NOT NULL,  
    status_funcionario VARCHAR(15) DEFAULT 'Ativo'  
);
```

```
CREATE TABLE Mesa (  
    id_mesa INT AUTO_INCREMENT PRIMARY KEY,  
    numero_mesa INT UNIQUE NOT NULL,  
    capacidade_lugares INT NOT NULL,  
    status_mesa VARCHAR(20) DEFAULT 'Disponivel'  
);
```

```
CREATE TABLE Fornecedor (  
    id_fornecedor INT AUTO_INCREMENT PRIMARY KEY,  
    nome_fornecedor VARCHAR(100) NOT NULL,  
    cnpj VARCHAR(18) UNIQUE NOT NULL,  
    telefone_fornecedor VARCHAR(15)  
);
```

```
CREATE TABLE Produto (  
    id_produto INT AUTO_INCREMENT PRIMARY KEY,  
    nome_produto VARCHAR(100) NOT NULL,  
    categoria_produto VARCHAR(50),  
    preco_venda_tabela DECIMAL(10,2) NOT NULL  
);
```

```
CREATE TABLE Insumo (  
    id_insumo INT AUTO_INCREMENT PRIMARY KEY,  
    nome_insumo VARCHAR(100) NOT NULL,  
    qtd_estoque_atual DECIMAL(10,2) NOT NULL,  
    unidade_medida VARCHAR(10) NOT NULL, -- 'kg', 'un', 'litro'  
    preco_unitario_atual DECIMAL(10,2) NOT NULL  
);
```

```
CREATE TABLE Jogo (  
    id_jogo INT AUTO_INCREMENT PRIMARY KEY,  
    nome_jogo VARCHAR(100) NOT NULL,  
    categoria_jogo VARCHAR(50),  
    status_jogo VARCHAR(20) DEFAULT 'Disponivel',
```



```
    aluguel_hora_atual DECIMAL(10,2) NOT NULL
);
```

-- 2. TABELAS DEPENDENTES (COM CHAVES ESTRANGEIRAS)

```
CREATE TABLE Reserva (
    id_reserva INT AUTO_INCREMENT PRIMARY KEY,
    data_hora_reserva DATETIME NOT NULL,
    status_reserva VARCHAR(20) DEFAULT 'Agendada', -- 'Agendada', 'Concluida',
    'Cancelada'
    qtd_pessoas_esperadas INT NOT NULL,
    id_cliente INT,
    id_mesa INT,
    FOREIGN KEY (id_cliente) REFERENCES Cliente(id_cliente),
    FOREIGN KEY (id_mesa) REFERENCES Mesa(id_mesa)
);
```

```
CREATE TABLE Compra_Estoque (
    id_compra INT AUTO_INCREMENT PRIMARY KEY,
    data_compra DATE NOT NULL,
    valor_total_nota DECIMAL(10,2) NOT NULL,
    id_fornecedor INT,
    FOREIGN KEY (id_fornecedor) REFERENCES Fornecedor(id_fornecedor)
);
```

```
CREATE TABLE Comanda (
    id_comanda INT AUTO_INCREMENT PRIMARY KEY,
    data_abertura DATETIME NOT NULL,
    valor_total_consumo DECIMAL(10,2) DEFAULT 0.00,
    gorjeta DECIMAL(10,2) DEFAULT 0.00,
    status_comanda VARCHAR(20) DEFAULT 'Aberta',
    desconto_percentual DECIMAL(5,2) DEFAULT 0.00,
    id_cliente INT,
    id_mesa INT,
    id_funcionario INT,
    FOREIGN KEY (id_cliente) REFERENCES Cliente(id_cliente),
    FOREIGN KEY (id_mesa) REFERENCES Mesa(id_mesa),
    FOREIGN KEY (id_funcionario) REFERENCES Funcionario(id_funcionario),
    -- SEU BLOQUEIO DE SEGURANÇA AQUI: impede descontos absurdos
    CONSTRAINT chk_desconto CHECK (desconto_percentual BETWEEN 0.00 AND
100.00)
);
```

-- 3. TABELAS PONTE (RELACIONAMENTOS N:M)

-- Ligação Comanda -> Produto

```
CREATE TABLE Item_Comanda_Produto (  
    id_comanda INT,  
    id_produto INT,  
    quantidade_pedida_produto INT NOT NULL,  
    preco_un_cobrado_produto DECIMAL(10,2) NOT NULL,  
    PRIMARY KEY (id_comanda, id_produto),  
    FOREIGN KEY (id_comanda) REFERENCES Comanda(id_comanda),  
    FOREIGN KEY (id_produto) REFERENCES Produto(id_produto)  
);
```

-- Ligação Comanda -> Insumo (Adicionais)

```
CREATE TABLE Item_Comanda_Insumo (  
    id_item_insumo INT AUTO_INCREMENT PRIMARY KEY, -- PK própria para permitir  
    múltiplos adicionais iguais  
    id_comanda INT,  
    id_insumo INT,  
    quantidade_adicional_insumo INT NOT NULL,  
    preco_un_cobrado_insumo DECIMAL(10,2) NOT NULL,  
    referencia_item_pai INT NULL, -- Guarda o ID do lanche onde esse adicional vai entrar  
    FOREIGN KEY (id_comanda) REFERENCES Comanda(id_comanda),  
    FOREIGN KEY (id_insumo) REFERENCES Insumo(id_insumo)  
);
```

-- Ligação Comanda -> Jogo (Aluguel)

```
CREATE TABLE Item_Comanda_Jogo (  
    id_comanda INT,  
    id_jogo INT,  
    hora_inicio_aluguel DATETIME NOT NULL,  
    hora_devolucao_aluguel DATETIME,  
    aluguel_hora_cobrado DECIMAL(10,2) NOT NULL,  
    PRIMARY KEY (id_comanda, id_jogo, hora_inicio_aluguel),  
    FOREIGN KEY (id_comanda) REFERENCES Comanda(id_comanda),  
    FOREIGN KEY (id_jogo) REFERENCES Jogo(id_jogo)  
);
```

-- Ficha Técnica: Ligação Produto -> Insumo (Ingredientes de fabricação)

```
CREATE TABLE Ficha_Tecnica_Produto (  
    id_produto INT,  
    id_insumo INT,  
    quantidade_necessaria DECIMAL(10,2) NOT NULL,  
    PRIMARY KEY (id_produto, id_insumo),  
    FOREIGN KEY (id_produto) REFERENCES Produto(id_produto),  
    FOREIGN KEY (id_insumo) REFERENCES Insumo(id_insumo)  
);
```

-- Ligação Compra_Estoque -> Insumo (Itens da Nota Fiscal)

```

CREATE TABLE Item_Compra_Insumo (
    id_compra INT,
    id_insumo INT,
    quantidade_comprada DECIMAL(10,2) NOT NULL,
    preco_custo_unitario DECIMAL(10,2) NOT NULL,
    PRIMARY KEY (id_compra, id_insumo),
    FOREIGN KEY (id_compra) REFERENCES Compra_Estoque(id_compra),
    FOREIGN KEY (id_insumo) REFERENCES Insumo(id_insumo)
);

```

Inserção

-- Clientes

```

INSERT INTO Cliente (nome_completo, cpf, telefone, email)
VALUES ('João Silva', '111.111.111-11', '65999999999', 'joao@email.com'),
      ('Maria Oliveira', '222.222.222-22', '65888888888', 'maria@email.com');

```

-- Funcionários

```

INSERT INTO Funcionario (nome_funcionario, cpf_funcionario, cargo, data_admissao,
status_funcionario)
VALUES ('Carlos Garçon', '333.333.333-33', 'Garçon', '2025-01-10', 'Ativo');

```

-- Mesas

```

INSERT INTO Mesa (numero_mesa, capacidade_lugares, status_mesa)
VALUES (1, 4, 'Disponivel'), (2, 6, 'Disponivel');

```

-- Produtos e Insumos

```

INSERT INTO Produto (nome_produto, categoria_produto, preco_venda_tabela)
VALUES ('Hamburguer Gamer', 'Lanche', 35.00);

```

```

INSERT INTO Insumo (nome_insumo, qtd_estoque_atual, unidade_medida,
preco_unitario_atual)
VALUES ('Bacon Extra', 50.00, 'un', 5.00);

```

-- Jogos

```

INSERT INTO Jogo (nome_jogo, categoria_jogo, status_jogo, aluguel_hora_atual)
VALUES ('Catan', 'Estratégia', 'Disponivel', 15.00);

```

-- Abrindo uma Comanda

```

INSERT INTO Comanda (data_abertura, valor_total_consumo, gorjeta, status_comanda,
desconto_percentual, id_cliente, id_mesa, id_funcionario)
VALUES ('2026-07-09 19:00:00', 40.00, 4.00, 'Aberta', 0.00, 1, 1, 1);

```

6. Aplicação de normalização e desnormalização

1. Processo de Normalização (Evolução V1 para V2)

- **Eliminação de Tabelas Híbridas e Anomalias (1FN e 2FN)**
 - **O Problema na V1:** Existia uma tabela chamada **Fluxo_Caixa** que misturava Chaves Estrangeiras de **Comanda** (Receitas), **Funcionario** (Despesas trabalhistas) e **Ingrediente** (Custos). Isso violava a normalização, pois forçava a criação de registros cheios de campos nulos (ex: uma compra de ingrediente deixava o campo da comanda vazio) e misturava regras de negócios completamente diferentes.
 - **A Solução na V2:** A tabela **Fluxo_Caixa** foi **excluída**. Entendemos que o lucro não deve ser armazenado estaticamente, mas sim calculado de forma dinâmica. As compras de estoque ganharam uma estrutura própria (**Compra_Estoque** e **Fornecedor**), e os relatórios financeiros passaram a ser gerados via consultas (**SELECT** com cálculos).
- **Resolução de Relacionamentos Ocultos (Muitos-para-Muitos)**
 - **O Problema na V1:** A tabela **Comanda** possuía apenas **Valor_Total** e **Taxa_Ludoteca**, sem nenhuma forma de registrar *quais* produtos ou jogos o cliente consumiu. Faltava a atomicidade exigida pela Primeira Forma Normal (1FN).
 - **A Solução na V2:** Foram criadas as entidades associativas (Tabelas Ponte) **Item_Comanda_Produto**, **Item_Comanda_Jogo** e **Item_Comanda_Insumo**. Agora, uma única comanda pode receber múltiplos itens individualizados, registrando o preço exato cobrado no momento do pedido.
- **Reestruturação de Chaves Primárias e Dependências (3FN)**
 - **O Problema na V1:** Tabelas como **Cliente** e **Funcionario** usavam o **CPF** como Chave Primária (PK). Além disso, a **Comanda** dependia diretamente da **Reserva**, o que impedia o atendimento de clientes que chegassem ao local sem agendamento prévio.
 - **A Solução na V2:** Substituímos os CPFs por Chaves Artificiais/Surrogate Keys (**id_cliente**, **id_funcionario**), melhorando a performance dos JOINS e evitando problemas de atualização em cascata. Desvinculamos a Comanda da Reserva; agora, ambas se ligam independentemente à **Mesa** e ao **Cliente**. Além disso, a tabela **Reserva** ganhou atributos de gestão de estado (**status_reserva**), permitindo o controle real de cancelamentos e confirmações.

2. Justificativa de Desnormalização Consciente

Embora a nova arquitetura (V2) siga a Terceira Forma Normal (3FN), exigindo junções (JOINS) para cruzar dados, aplicamos a **desnormalização estratégica** em dois pontos

críticos para garantir alta performance e tempo de resposta rápido nas operações do dia a dia do estabelecimento:

- **Tabela Comanda - Coluna valor_total_consumo:** * Pela regra estrita de normalização, o valor total é um atributo derivado (soma dos itens consumidos) e não deveria ser armazenado. Contudo, decidimos manter a coluna `valor_total_consumo` física na comanda. Essa desnormalização evita que o banco tenha que reprocessar a soma de milhares de linhas das tabelas associativas toda vez que o caixa precisar visualizar a tela de comandas ativas. O valor é atualizado automaticamente por meio de *Triggers*.
- **Tabela Insumo - Coluna qtd_estoque_atual:**
 - O estoque real poderia ser descoberto dinamicamente subtraindo as saídas (itens vendidos baseados na ficha técnica) das entradas (compras). No entanto, para evitar consultas pesadas de inventário, adicionamos a coluna de saldo atual (`qtd_estoque_atual`). Trata-se de uma desnormalização controlada por funções do banco (`dar_baixa_estoque_venda`), resultando em um ganho massivo de performance.

7. String de conexão no Postgresql (Neon)

postgresql://neondb_owner:npg_4BSyQCjL0ltb@ep-hidden-sunset-acc0dskn.sa-east-1.aws.
[neon.tech](https://neon.tech/neondb?sslmode=require&channel_binding=require)/neondb?sslmode=require&channel_binding=require

8 & 9. Relatórios

Relatórios com mais de 5 tabelas:

Relatório 1: Extrato Completo de Comandas (Número comanda, Cliente, Garçon, Status da comanda, Item consumido, valor total)

-- Parte 1: Puxa o consumo de Produtos/Lanches

```
SELECT
  co.id_comanda AS "Num. Comanda",
  c.nome_completo AS "Cliente",
  m.numero_mesa AS "Mesa",
  f.nome_funcionario AS "Atendente",
  co.status_comanda AS "Status da Comanda",
  'Produto Físico' AS "Categoria",
  p.nome_produto AS "Item Consumido/Alugado",
  icp.quantidade_pedida_produto AS "Qtd",
  (icp.quantidade_pedida_produto * icp.preco_un_cobrado_produto) AS "Valor Total Linha"
FROM Comanda co
INNER JOIN Cliente c ON co.id_cliente = c.id_cliente
INNER JOIN Mesa m ON co.id_mesa = m.id_mesa
```

```
INNER JOIN Funcionario f ON co.id_funcionario = f.id_funcionario
INNER JOIN Item_Comanda_Produto icp ON co.id_comanda = icp.id_comanda
INNER JOIN Produto p ON icp.id_produto = p.id_produto
```

UNION ALL

-- Parte 2: Puxa o aluguel de Jogos e empilha embaixo

```
SELECT
    co.id_comanda AS "Num. Comanda",
    c.nome_completo AS "Cliente",
    m.numero_mesa AS "Mesa",
    f.nome_funcionario AS "Atendente",
    co.status_comanda AS "Status da Comanda",
    'Entretenimento (Jogo)' AS "Categoria",
    j.nome_jogo AS "Item Consumido/Alugado",
    1 AS "Qtd", -- Considera 1 locação por registro
    icj.aluguel_hora_cobrado AS "Valor Total Linha"
FROM Comanda co
INNER JOIN Cliente c ON co.id_cliente = c.id_cliente
INNER JOIN Mesa m ON co.id_mesa = m.id_mesa
INNER JOIN Funcionario f ON co.id_funcionario = f.id_funcionario
INNER JOIN Item_Comanda_Jogo icj ON co.id_comanda = icj.id_comanda
INNER JOIN Jogo j ON icj.id_jogo = j.id_jogo
ORDER BY "Num. Comanda", "Categoria";
```

Relatório 2: Consumo de Insumos (Estoque) por Comanda

```
SELECT
    co.id_comanda AS "Num. Comanda",
    p.nome_produto AS "Lanche Vendido",
    icp.quantidade_pedida_produto AS "Qtd Lanches",
    i.nome_insumo AS "Ingrediente Usado",
    (ft.quantidade_necessaria * icp.quantidade_pedida_produto) AS "Total Insumo
Descontado",
    i.unidade_medida AS "Medida"
FROM Comanda co
INNER JOIN Item_Comanda_Produto icp ON co.id_comanda = icp.id_comanda
INNER JOIN Produto p ON icp.id_produto = p.id_produto
INNER JOIN Ficha_Tecnica_Produto ft ON p.id_produto = ft.id_produto
INNER JOIN Insumo i ON ft.id_insumo = i.id_insumo
WHERE co.id_comanda = 1; -- ID da comanda que deseja investigar
```

Relatório 3: Histórico de Locação de Entretenimento

```
SELECT
```

```

    c.nome_completo AS "Cliente",
    m.numero_mesa AS "Mesa Utilizada",
    j.nome_jogo AS "Jogo Alugado",
    icj.hora_inicio_aluguel AS "Início do Aluguel",
    icj.hora_devolucao_aluguel AS "Devolução",
    icj.aluguel_hora_cobrado AS "Valor Cobrado"
FROM Comanda co
INNER JOIN Cliente c ON co.id_cliente = c.id_cliente
INNER JOIN Mesa m ON co.id_mesa = m.id_mesa
INNER JOIN Item_Comanda_Jogo icj ON co.id_comanda = icj.id_comanda
INNER JOIN Jogo j ON icj.id_jogo = j.id_jogo
ORDER BY c.nome_completo;

```

Outros relatórios:

Agenda de Ocupação e Reservas

```

SELECT
    r.data_hora_reserva AS "Data e Hora do Agendamento",
    c.nome_completo AS "Nome do Cliente Titular",
    c.telefone AS "Telefone de Contato",
    r.qtd_pessoas_esperadas AS "Pessoas Esperadas",
    m.numero_mesa AS "Mesa Designada",
    m.capacidade_lugares AS "Capacidade da Mesa",
    r.status_reserva AS "Status Atual",
    CASE
        WHEN r.qtd_pessoas_esperadas > m.capacidade_lugares THEN 'Alerta: Faltam
lugares'
        ELSE 'Capacidade OK'
    END AS "Status de Capacidade"
FROM Reserva r
INNER JOIN Cliente c ON r.id_cliente = c.id_cliente
INNER JOIN Mesa m ON r.id_mesa = m.id_mesa
ORDER BY r.data_hora_reserva;

```

Demonstrativo Financeiro Mensal (Lucro Líquido)

```

WITH Faturamento AS (
    -- Soma as comandas fechadas aplicando o desconto percentual
    SELECT SUM(valor_total_consumo - (valor_total_consumo * (desconto_percentual /
100))) AS receita_liquida
    FROM Comanda
    WHERE status_comanda = 'Fechada'
    AND EXTRACT(MONTH FROM data_abertura) = 7
    AND EXTRACT(YEAR FROM data_abertura) = 2026
),

```

```

Custos_Estoque AS (
  -- Soma todas as notas fiscais de compras
  SELECT SUM(valor_total_nota) AS total_compras
  FROM Compra_Estoque
  WHERE EXTRACT(MONTH FROM data_compra) = 7
        AND EXTRACT(YEAR FROM data_compra) = 2026
),
Custos_Trabalhistas AS (
  -- Soma os salários dos funcionários que estão ativos
  SELECT SUM(salario) AS folha_pagamento
  FROM Funcionario
  WHERE status_funcionario = 'Ativo'
)
SELECT
  COALESCE(f.receita_liquida, 0) AS "Faturamento Limpo (R$)",
  COALESCE(ce.total_compras, 0) AS "Despesas com Fornecedores (R$)",
  COALESCE(ct.folha_pagamento, 0) AS "Despesas Folha de Pagamento (R$)",
  (COALESCE(f.receita_liquida, 0) - COALESCE(ce.total_compras, 0) -
   COALESCE(ct.folha_pagamento, 0)) AS "Lucro Líquido Final (R$)"
FROM Faturamento f
CROSS JOIN Custos_Estoque ce
CROSS JOIN Custos_Trabalhistas ct;

```

Otimização do relatório: “ Relatório 1: Extrato Completo de Comandas (Número comanda, Cliente, Garçom, Status da comanda, Item consumido, valor total)”

```

WITH Itens_Consumidos AS (
  -- Pega apenas os dados de Produtos
  SELECT
    id_comanda,
    'Produto Físico' AS Categoria,
    p.nome_produto AS Item_Nome,
    icp.quantidade_pedida_produto AS Qtd,
    (icp.quantidade_pedida_produto * icp.preco_un_cobrado_produto) AS
Valor_Total_Linha
  FROM Item_Comanda_Produto icp
  INNER JOIN Produto p ON icp.id_produto = p.id_produto

  UNION ALL

  -- Pega apenas os dados de Jogos e empilha
  SELECT

```



```

        id_comanda,
        'Entretenimento (Jogo)' AS Categoria,
        j.nome_jogo AS Item_Nome,
        1 AS Qtd,
        icj.aluguel_hora_cobrado AS Valor_Total_Linha
    FROM Item_Comanda_Jogo icj
    INNER JOIN Jogo j ON icj.id_jogo = j.id_jogo
)
-- Agora faz o JOIN com as tabelas pesadas (Cliente, Funcionario, Mesa)
SELECT
    co.id_comanda AS "Num. Comanda",
    c.nome_completo AS "Cliente",
    m.numero_mesa AS "Mesa",
    f.nome_funcionario AS "Atendente",
    co.status_comanda AS "Status da Comanda",
    ic.Categoria AS "Categoria",
    ic.Item_Nome AS "Item Consumido/Alugado",
    ic.Qtd AS "Qtd",
    ic.Valor_Total_Linha AS "Valor Total Linha"
FROM Comanda co
INNER JOIN Cliente c ON co.id_cliente = c.id_cliente
INNER JOIN Mesa m ON co.id_mesa = m.id_mesa
INNER JOIN Funcionario f ON co.id_funcionario = f.id_funcionario
INNER JOIN Itens_Consumidos ic ON co.id_comanda = ic.id_comanda
ORDER BY co.id_comanda, ic.Categoria;

```

10. Implementação de Dados complementares (NoSql)

Uso de Dados Complementares (Abordagem NoSQL com JSONB)

Para cumprir o requisito de armazenamento de dados complementares sem a necessidade de uma API externa,

o projeto utiliza o suporte nativo do PostgreSQL a dados não-estruturados por meio do tipo **JSONB**.

A tabela criada para essa finalidade é a **Recordacoes_Sessao_NoSQL**, cujo objetivo é registrar o histórico de jogatinas das mesas e uma nota da experiência que tiveram do local de 0 a 5 estrelas.

Justificativa do Modelo NoSQL para Recordação de Sessões

A tabela **Recordacoes_Sessao_NoSQL** foi projetada para funcionar como um repositório de memórias das experiências dos clientes no estabelecimento. Quando um grupo de clientes finaliza sua visita, eles podem registrar um review detalhado sobre os jogos jogados, placares marcantes, notas e observações descontraídas sobre aquela noite.

- **Por que NoSQL (JSONB) é ideal aqui?** Porque o formato de uma noite de jogos é totalmente imprevisível. Um grupo pode jogar três jogos rápidos e querer registrar quem foi o "pior jogador", enquanto outro grupo pode jogar um único RPG de 4 horas e querer anexar um resumo da história da campanha. Tentar engessar essas interações humanas em colunas relacionais fixas destruiria a flexibilidade da ferramenta.
- **Por que está atrelado à **id_comanda**?** A comanda funciona como o "token" que resume aquela noite específica (data, mesa, quem atendeu e o consumo). Atrelando o JSON à comanda, permitimos que, em uma visita futura, o sistema busque retroativamente os registros das comandas passadas daquele cliente, trazendo de volta suas memórias e gerando um forte sentimento de nostalgia e fidelização.
- **Dado Complementar (Fora do DER):** Esta estrutura não consta no Diagrama de Entidade-Relacionamento original por ser uma extensão de dados desestruturados (schemaless). O campo JSONB funciona como uma "caixa-preta" onde a aplicação tem total liberdade para gravar novas propriedades sem necessidade de alteração na estrutura física do banco relacional.

Código de Criação e inserção

```
CREATE TABLE Recordacoes_Sessao_NoSQL (  
    id_registro SERIAL PRIMARY KEY,  
    id_comanda INT REFERENCES Comanda(id_comanda),  
    nota_experiencia INT CHECK (nota_experiencia BETWEEN 0 AND 5), -- Coluna SQL  
    para relatórios de média  
    data_registro TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    dados_review JSONB -- Coluna NoSQL para a bagunça dos jogos  
);  
  
INSERT INTO Recordacoes_Sessao_NoSQL (id_comanda, nota_experiencia,  
dados_review) VALUES  
(1, 5, '{
```

```
"jogo_jogado": "Catan",

"resumo_da_noite": "Noite memorável! O clima do café estava ótimo e a mesa de
tabuleiros bem iluminada.",

"vencedor": "Carlos",

"detalhes_da_partida": {

    "duracao_minutos": 115,

    "maior_estrada": "Mariana",

    "maior_exercito": "João",

    "recurso_mais_escaso": "Tijolo"

},

"classificacao_final": [

    {"jogador": "Lucas", "pontos": 10, "estrategia": "Foco em Cidades e Cartas de
Desenvolvimento"},

    {"jogador": "Mariana", "pontos": 9, "estrategia": "Expansão de Estradas e Portos de
Trigo"},

    {"jogador": "Felipe", "pontos": 6, "estrategia": "Ficou preso no deserto por falta de
ovelhas"}

],

"memorias_para_o_futuro": "O Lucas ganhou de virada na última rodada comprando uma
carta de ponto de vitória escondido. A Mariana quase virou a mesa. Precisamos voltar para
revanche!"

});
```

